# 최종 설계 보고서

대학교에서 겹치는 강의를 듣는 친구 찾아주기 서비스 '겹강이' 데이터베이스 설계

제출일: 2024년 6월 10일

정경대학 미디어학과 전지원(2022103086)

이과대학 생물학과 권혜빈(2021102426)

# 1. 설계 배경

## 1.1 프로젝트의 중요성/필요성

재미있고 기억에 남을 대학생활에서 친구는 필수적 요소지만, 모두가 친구가 많은 것은 아니다. 수업을 같이 들을 친구, 조별과제를 같이 할 친구, 더 나아가 밥을 같이 먹거나 함께 공부할 친구 등 대학생활을 함께 할 친구를 찾는 것은 쉽지 않다. 다전공생이거나 복학생, 고학번 학생 등 아는 사람이 없어서 또는 내향적인 성격을 가져서, 대형 강의라 친구를 사귀기 어려운 경우 등 다양한 이유가 있다. 우리 조는 대학 생활에서 새로운 친구를 사귀는 가장 쉬운 방법은 같은 수업을 듣는 친구를 사귀는 것으로 보고 프로젝트를 진행하였다.
사전에 진행한 혼강(혼자 강의를 듣는 것)에 대한 설문조사에서는 약 45%의 응답자가 혼강이 '그럭저럭 들을 만하다'고 응답하였으나 대부분의 응답자가 혼강의 장점보다는 단점에 더 공감을 많이 하는 결과를 보였다. 혼강의 단점에 대해서는 '수업에 빠졌을 때 전달 사항을 알기 어렵다', '수업 후 족보를 구하거나 수업 관련 팁을 공유할 사람이 없다' 선지가 응답률이 가장 높았고, 그 외에도 '조별 과제 시 팀원을 구하기 어렵다' 등이 혼강의 단점으로 선택되었다. 이에 따라 우리 조는 겹강 친구, 즉 겹치는 강의를 듣는 친구를 찾아주는 앱을 제작해, 이용자가 자신의 시간표를 입력하면 이를 바탕으로 다른 이용자들의 시간표와 비교해 같은 강의를 듣는 이용자끼리 연결해 주는 앱을 기획했고 이를 위한 DB설계를 수행할 예정이다.

## 1.2 문제정의

겹강 친구를 찾고 싶은 이용자는 회원가입 후 프로필에 학번, 성별, 자기소개 등 프로필 입력이 가능하고, 본인의 시간표를 등록, '친구 찾기' 기능을 통해 해당 이용자와 같은 강의를 듣는 다른 이용자들이 매칭된다. 이때, '최소 겹강 개수'나 '최대 겹강 개수' 등 세부 친구 찾기 조건들을 설정할 수 있으며 '친구 찾기' 기능을 통해 매칭된 다른 이용자들에게 친구 신청을 할 수 있다. 친구 신청을 받은 이용자가 수락을 해야만 신청한 이용자가 상대의 프로필의 개인정보(성별, 학번 등)을 확인할 수 있으며, 친구 수락 전까지는 보이지 않는다.

- 회원 가입 기능
  - ID(중복 방지), PW 설정
  - 이메일(중복 방지), 사용자 이름(본명), 닉네임(중복방지) 입력

- 시간표 등록 기능
  - 종합강의시간표 조회
  - 강의명으로 강의 검색해서 이용자의 시간표에 추가 가능

- 내 프로필 기능
  - 이용자가 회원가입후 프로필 등록 가능
  - 이때 자기소개, 성별, 학번, 전공 등 입력 가능하며 해당 항목을 친구 공개로 할 것인지 전체 공개로 할 것인지 선택 가능함.

- 친구 찾기 기능
  - 이용자의 시간표를 바탕으로 겹치는 강의가 있는 다른 이용자들을 찾아줌.
  - 친구 찾기로 매칭된 다른 이용자에게 친구 신청을 할 수 있음.

- 친구 리스트, 채팅 기능
  - '친구 리스트'에서 현재 친구인 회원들을 확인할 수 있음
  - '채팅'에서 현재 친구인 회원들에게 메시지를 보낼 수 있음

### 1.3 프로젝트 완성 후의 기대 효과

'겹강이' 서비스를 통해 이용자들은 친구를 만들고, 같은 강의를 듣는 학우들과 학습 계획을 세우고, 함께 공부를 하거나 팀 과제를 함께 하는 등 교우관계 증진과 학업 성취도 향상의 일석이조의 효과를 기대할 수 있을 것이다.

### 1.4 기타, 설계 계획서의 내용 요약

파이썬 기반의 Flask에 sqlite3을 DBMS로 사용해 서비스를 설계할 예정이다. 또한 MVC모델을 이용, pythonanywhere 웹 호스팅을 이용하고 react-native로 Web-view 앱으로 제작할 예정이다.

## 2. 설계 목표

### 2.1 요구 사항 등을 분석 정리한 내용 요약

겹강 친구를 찾아주는 서비스를 설계하므로 회원, 강의, 회원 정보, 친구 등의 엔티티를

ER다이어그램을 통해 정의한 후 기능에 필요한 관계를 정의할 예정이다. 또한 사용자에게 편리성을 제공하는 UI/UX를 통해 어플리케이션을 이용할 수 있어야 한다. 현재 필요한 기능을 간단히 요약하자면 다음과 같다.

- 회원가입(이때 id, email, 닉네임은 기존 회원과 중복되지 않아야 함), 로그인
- 강의 목록(종합강의시간표)를 통해 과목 조회, 시간표에 추가, 삭제
- '친구 찾기'에서 같은 강의를 듣는 이용자와 매칭 기능, 친구 신청
- '알림'을 통해 다른 이용자의 친구 신청 수락 또는 거절
- '친구 목록' 확인, 친구와 '채팅' 기능
- '내 프로필'에서 이용자 프로필 수정, 등록 가능(이때 id, email, 사용자 이름은 변경 불가능함)

## 2.2 설계 과정에서 주요하게 고려하여야 할 사항들 정리

1)MEMBERS 엔티티는 '겹강이' 서버 접속시 사용자가 된다. 그래서 회원가입 시 MEMBERS릴레이션의 로그인을 위해 필요한 아이디(MEMBERS.ID)와 비밀번호(MEMBERS.PW), 이메일주소(MEMBERS.EMAIL), 본명(MEMBERS.REAL_NAME), 닉네임(MEMBERS.NICK_NAME)를 입력하게 된다. ID, PW, EMAIL, REAL_NAME, NICK_NAME을 입력하지 않을 시 '겹강이'서비스를 이용하지 못하므로 이 애트리뷰트들은 절대 널값을 가질 수 없다. 반면 MEMBERS릴레이션에 INTRO, YEAR, SEX, MAJOR는 로그인 이후 '내프로필'페이지(http://127.0.0.1:5000/myprofile)로 이동하여 추가적으로 자신의 정보를 입력할 수 있다. INTRO는 한줄소개로 데이터타입을 text로 받아 자유롭게 자기소개를 기입할 수 있다. YEAR은 학번으로 integer타입으로 받는다. SEX, MAJOR1, MAJOR2는 각각 성별, 전공이다. (특히 MAJOR을 제외한 MEMBERS의 애트리뷰트는 모두 단일 애트리뷰트인데, 복합 애트리뷰로 MAJOR를 가질 수 있다. MAJOR의 개별 애트리뷰트가 MAJOR1, MAJOR2 2개로 이뤄진다. 보통의 단일전공의 경우 MAJOR1값에 정상적으로 자신의 전공이름이 쓰여지고, MARJOR2는 널값이다. 다전공자의 경우 MAJOR값을 예를 들어 '생물학과', '컴퓨터 공학과' 2개의 값을 가질 수 있다. 이처럼 MAJOR1은 절대 널값이 나올 수 없지만 MAJOR2는 널값이 가능하다) ID, PW, EMAIL, REAL_NAME, NICK_NAME와 더불어 SEX, MAJOR_1, MAJOR_2의 데이터 타입은 모두 text이다. MEMBERS의 primary key는 ID이다.

2)TOTALLECUTRES 엔티티는 경희대학교 국제캠퍼스 2024-1학기 기준 종합시간표에 있는 모든 강의 정보가 된다. TOTALLECUTRES는 ID, NAME, PROF, CREDIT, START_TIME,

END_TIME, DAY_OF_WEEK, YEAR의 애트리뷰트를 가진다. ID는 강의의 학수번호이고 text타입이다. 학수번호는 고유하므로 기본 키로 사용가능하다. NAME은 강의명(text타입), PROF은 교수명(text타입), CREDIT은 그 강의를 수강시 취득할 수 있는 학점 수이고, integer데이터 타입이다. START_TIME은 강의시작시간, END_TIME은 강의끝나는시간으로, 역시 둘다 text타입이다. DAY_OF_WEEK은 요일을 의미하고(text타입), YEAR은 대상학년으로 integer타입이다. 이때 DAY_OF_WEEK를 제외한 나머지 모든 TOTALLECTURES의 애트리뷰트는 단일 애트리뷰트인 반면, DAY_OF_WEEK는 예를 들어 '화요일'과 '목요일' 2개의 값을 가질 수 있기 때문에 다치 애트리뷰트에 속한다.

3)FRIEND 엔티티는 현 사용자가 '겹강친구 찾기' 알고리즘을 통해 알게 된 같은 강의를 수강하는 다른 사용자들을 의미한다. 현 사용자가 '겹강친구 찾기' 알고리즘을 통해 매칭된 결과를 다른 사용자들의 이름을 나열하여 알려주는데, 현 사용자를 USER1이라 하고 다른 사용자를 USER2라 할 때, FRIEND릴리이션에 {USER1, USER2, FRIEND_STATUS=0}인 튜플이 새로 생성된다. USER1과 USER2 둘다 기본키가 되어야 하므로 FRIEND는 결국 USER1, USER2를 복합키로 갖는다. 동시에 USER1과 USER2는 MEMBERS.ID 즉, MEMBERS의 기본키를 외래키로 갖는다. 따라서 USER1, USER2는 text타입을 데이터로 갖고, FRIEND_STATUS는 2명의 사용자 사이의 친구관계를 0또는 1또는 2인 세가지 상태로 분류하여 정의하였다. 따라서 FRIEND_STATUS값은 integer타입이다. FRIEND_STATUS=0은 '겹강친구 매칭'을 진행한 직후 자동으로 투플이 생성되고, FRIEND_STATUS=1은 USER1이 USER2에게 '친구요청'을 신청했지만 USER2가 '수락'하지 않은 상태를 의미한다. 최종적으로 USER2가 로그인 이후의 화면에 최초 접속시 USER1으로부터 온 모달창이 나타난다. 모달창에는 '수락'과 '보류'버튼이 있는데 수락버튼을 누를 시 USER1과 USER2사이의 FRIEND_STATUS=1-> FRIEND_STATUS=2로 즉시 전환되면서 서로 간의 실시간 일대일 채팅이 가능해진다. '보류'버튼을 누를 시에는

FRIEND_STATUS=1그대로 유지되면서 실질적으로 '보류'버튼은 모달창을 닫는 기능만 수행할 뿐이다.

4)다음은 NOTIFICATION관계에 대해 설명이다. NOTIFICATION {ID, USER_ID, SENDER_ID, SEEN, MESSAGE}다음과 같은 애트리뷰트를 갖는다. MEMBERS와 FRIEND릴레이션을 'FRIEND.USER1=NOTIFICATION.SENDER_ID 그리고 FRIEND.USER2=NOTIFICATION.USER_ID' 다음과 같은 조건으로 조인시켜야한다. 따라서 NOTIFICATION의 SENDER_ID는 FRIEND의 USER1를 외래키로 갖고 NOTIFICATION의 USER_ID는 FRIEND의 USER2를 외래키로 갖는다. 각 알림의 ID는 고유할 테니 NOTIFICATION의 기본키이다. SEEN애트리뷰트를 제외한 ID, USER_ID, SENDER_ID, MESSAGE애트리뷰트는 text데이터 유형이고, SEEN은 0또는 1만의 값을 지니므로 INTEGER(더 나아가 BOOLEAN값이 된다.)

현 사용자가 다른 사용자들에게 '친구 요청' 버튼을 눌러 친구가 되기를 요청한다면 FRIEND_STATUS=0---> FRIEND_STATUS=1로 전환된다. 동시에 NOTIFICATION 관계는 USER1이 USER2에게 'NAME님의 새로운 친구요청이 있습니다.'라고 NOTIFICATION을 보내게 된다. 이때의 텍스트 내역은 NOTIFICATION릴레이션의 MESSAGE애트리뷰트에 들어가고, NAME은 FRIEND.USER1와 MEMBERS.ID를 조인 시켰을 때 MEMBERS의 REAL_NAME에서 가져온 이름이다. 여기서 현사용자인 USER1서버를 로그아웃하고 다른 사용자였던 USER2에 로그인을 하여 hello화면에 최초 접속 시 방금 NOTIFICATION의 MESSAGE과 '수락'과 '보류' 버튼이 모달창에 함께 생성되는 것을 확인할 수 있다. '수락' 버튼을 누를 시 FRIEND_STATUS=2로 전환되는 동시에, 이번에는 USER2가 USER1에게 NOTIFICATION의 MESSAGE로 'NAME님이 친구 요청을 수락하였습니다.'라고 USER1의 hello화면에 띄우게 된다. 이때 NAME은 NOTIFICATION, FREIND, MEMBERS 세 개를 조인했을 때 MEMBERS.id= FRIEND.user1= NOTIFICATION.sender_id,, MEMBERS.NAME이 된다.

5)MESSAGE관계는 ID, SENDER_ID, RECIPIENT_ID, MESSAGE, TIMESTAMP 애트리뷰트를 가진다. ID를 통해 각 메시지의 구분이 가능하므로 ID는 MESSAGE의 기본키이고, integer데이터 타입을 가진다. SENDER_ID, RECIPIENT_ID, MESSAGE, TIMESTAMP는 모두 text데이터 타입을 가진다. SENDER_ID와 RECIPIENT_ID는 MEMBERS의 기본키인 ID를 외래키로 가진다.

6)마지막 관계인 LECTURES는 USER_ID와 SUBJ_ID를 애트리뷰트로 갖는 릴레이션이다. LECTURES의 USER_ID는 MEMBERS의 ID를 외래키로 갖고, MEMBERS의 SUBJ_ID는 TOTALLECTURES의 ID를 외래키로 갖는다. 때문에 USER_ID와 SUBJ_ID는 모두 text타입의 데이터를 갖는다.

■ 각 엔티티와 엔티티 관계는 다음과 같이 연결된다.

1)MEMBERS<--->NOTIFICATION [1:N]관계

한 명의 MEMBER는 여러 개의 NOTIFICATION을 받을 수 있지만, 한 개의 NOTIFICATION에 대해서는 한 개의 MEMBERS가 projection하기 때문이다.

2)NOTIFICATION<--->FRIEND [N:1]관계

한 개의 FRIEND 관계는 여러 개의 NOTIFICATION을 생성할 수 있지만, 한 개의 NOTIFICATION에 대해서는 한 개의 MEMBERS에 맵핑된다.

3)MEMBERS<--->MESSAGE [1:N]관계

한 명의 MEMBERS는 여러 개의 MESSAGE를 보낼 수 있으며, 한 개의 MESSAGE를 한 명의 MEMBERS가 받는다.

4)MESSAGE<--->FRIEND [N:1]관계

한 개의 FRIEND 관계는 여러 개의 MESSAGE를 생성할 수 있다. 반면 MESSAGE는 FRIEND

한 명이 한 개씩 보낼 수 있다.

5) MEMBERS<--->LECTURES [M:N]관계

한 명의 MEMBER는 여러 개의 LECTURES를 수강할 수 있다. 한 개의 LECTURES에 대해서도 여러 명의 MEMBERS가 수강가능하다.

6)LECTURES<--->TOTALLECUTRES [N:1]관계

LECTURES는 MEMBERS의 ID와 TOTALLECTURES의 ID를 외래키로 가지므로, TOTALLECTURES에 한 개의 수강과목에서 MEMBERS의 ID가 여러 개 존재가능하므로 LECTURES의 튜플이 여러 개 매핑가능하다. LECTURES 한 개의 튜플은 오직 TOTALLECTURES의 하나와 매핑가능하다(SENDER_ID로 듣는 사람도 고유하게 처리하였기 때문에)

## 2.3 사용할 시스템 환경에 대한 소개

구축할 '겹강이' 친구찾기 서비스는 클라이언트-서버 기반의 웹 환경에서 사용할 수 있다. 이에 따른 기술 스택 및 사용 플랫폼은 다음과 같다.

| | |
|---|---|
| **FE** | HTML, CSS |
| **BE** | Python |
| **DATABASE** | Sqlite3 |
| **DESIGN PATTERN** | MVC |
| **OS** | Windows |

사용자는 웹페이지를 통해 어플리케이션을 사용하며, 프론트엔드는 html과 css를 이용, 백엔드는 파이썬을 이용하여 어플리케이션을 구축할 것이다.

Flask를 이용한 앱 개발이라 데이터베이스는 flask가 내장지원하는 Sqlite3을 사용한다. 채팅 기능을 위해 flask-socketio를 이용하고 pythonanywhere 웹 호스팅을 이용해 웹 서버 구현, react-native expo로 Web-view앱으로 제작할 것이다.

## 2.4 설계 구현된 시스템을 평가할 수 있는 항목의 제시

TDD(테스트 주도 개발) 방법을 이용할 것이다. 우선 어플리케이션 개발 전에 먼저 테스트 프로그램을 기능별로 작성한다. 이후 기능 개발이 완료되면 단위 테스트를 수행한다. 모든 기능이 개발이 완료되고 단위 테스트까지 통과되면 통합 테스트를 수행해야한다.

단위 테스트를 수행해야 할 기능들은 다음과 같다.

**회원(USER)**

- 회원가입, 로그인
- 프로필 등록/수정
- 시간표 등록/삭제
- 친구 찾기 돌리기
- 친구 신청/ 수락
- 다른 이용자 프로필 조회
- 친구 목록 확인
- 친구 채팅 메시지 주고받기

# 3. 설계 내용

## 3.1 앞에서 기술한 내용을 반영하는 개념적인 데이터베이스 설계(ER 다이어그램 작성)

Draw.io를 통해 작성한 ER 다이어그램은 다음과 같다.

## 3.2 데이터베이스 정리 요약 ER 다이어그램을 중심으로

[특이사항]

앞서 설명한 6개의 관계인

MEMBERS<--->NOTIFICATION

NOTIFICATION<--->FRIEND

MEMBERS<--->MESSAGE

MESSAGE<--->FRIEND

MEMBERS<--->LECTURES

LECTURES<--->TOTALLECUTRES 중에 오직 **MESSAGE<--->FRIEND**관계만 약한 엔티티타입 관계이다. 왜냐하면 MEMBERS의 유저는 FRIEND릴레이션의 FRIEND_STATUS=2에 속하는 유

저들과만 일대일 실시간 채팅을 이용할 수 있기 때문이다. 즉 FRIEND의 모든 튜플이 MESSAGE관계에 대해 전체 참여하는 것이 아니기 때문에 부분 참여가 된다. 나머지 5개의 관계는 각각의 릴레이션이 전체 참여하므로 강한 엔티티 타입이다.

MEMEBERS의 MAJOR는 MAJOR1, MAJOR2의 개별 애트리뷰트를 가지는 복합 애트리뷰트이고, TOTALLECTURES의 DAY_OF_WEEK는 최대 원자값을 2개 가질 수 있는 다치 애트리뷰트이다.

FRIEND의 FRIEND_STATUS는 전체 '겹강이'앱 알고리즘의 키가 되는 애트리뷰트이다. FRIEND_STATUS는 오직 {0,1,2}의 값 중의 하나를 가지며 MESSAGE와 NOTIFICATION의 기능을 조절할 수 있다.

## 3.3 앞에서 기술한 내용을 반영하는 논리적인 데이터베이스 설계(테이블 매핑)

Flask에서 Sqlite3을 사용하여 각각의 엔티티를 실제 DB속 테이블로 매핑 시키는 작업을 수행한다(ER-릴레이션 사상). 생성된 테이블은 다음과 같다.

**[MEMBERS엔티티 => MEMBERS테이블]**

| cid | name | type | notnull | dflt_value | pk |
|---|---|---|---|---|---|
| 0 | ID | TEXT | 0 | | 1 |
| 1 | PW | TEXT | 1 | | 0 |
| 2 | EMAIL | TEXT | 1 | | 0 |
| 3 | REAL_NAME | TEXT | 1 | | 0 |
| 4 | NICK_NAME | TEXT | 1 | | 0 |
| 5 | INTRO | TEXT | 0 | | 0 |
| 6 | YEAR | INTEGER | 0 | | 0 |
| 7 | SEX | TEXT | 0 | | 0 |
| 8 | MAJOR_1 | TEXT | 1 | | 0 |

| | | | | | |
|---|---|---|---|---|---|
| 9 | MAJOR_2 | TEXT | 0 | | 0 |

**[TOTALLECTURES엔티티 => TOTALLECTURES테이블]**

| cid | name | type | notnull | dflt_value | pk |
|---|---|---|---|---|---|
| 0 | ID | TEXT | 0 | | 1 |
| 1 | NAME | TEXT | 1 | | 0 |
| 2 | PROF | TEXT | 1 | | 0 |
| 3 | CREDIT | INTEGER | 1 | | 0 |
| 4 | START_TIME | TEXT | 1 | | 0 |
| 5 | END_TIME | TEXT | 1 | | 0 |
| 6 | YEAR | INTEGER | 1 | | 0 |

**[TOTALLECTURES의 다치 애트리뷰트 DAY_OF_WEEK => TOTALLECTURES_DAYS테이블]**

| cid | name | type | notnull | dflt_value | pk |
|---|---|---|---|---|---|
| 0 | LECTURE_ID | TEXT | 1 | | 1 |
| 1 | DAY_OF_WEEK | TEXT | 1 | | 2 |

**[LECTURES 관계타입 => LECTURES테이블]**

| cid | name | type | notnull | dflt_value | pk |
|---|---|---|---|---|---|
| 0 | USER_ID | TEXT | 1 | | 1 |
| 1 | SUBJ_ID | TEXT | 1 | | 2 |

**[FRIEND엔티티 => FRIEND테이블]**

| cid | name | type | notnull | dflt_value | pk |
|---|---|---|---|---|---|
| 0 | USER1 | TEXT | 1 | | 1 |
| 1 | USER2 | TEXT | 1 | | 2 |
| 2 | FRIEND_STATUS | INTEGER | 1 | | 0 |

**[MESSAGE 관계타입=> MESSAGE테이블]**

| cid | name | type | notnull | dflt_value | pk |
|---|---|---|---|---|---|
| 0 | ID | INTEGER | 0 | | 1 |
| 1 | SENDER_ID | TEXT | 1 | | 0 |
| 2 | RECIPIENT_ID | TEXT | 1 | | 0 |
| 3 | MESSAGE | TEXT | 1 | | 0 |
| 4 | TIMESTAMP | TEXT | 1 | | 0 |

**[NOTIFICATION 관계타입 => NOTIFICATION 테이블]**

| cid | name | type | notnull | dflt_value | pk |
|---|---|---|---|---|---|
| 0 | ID | INTEGER | 0 | | 1 |
| 1 | USER_ID | TEXT | 1 | | 0 |
| 2 | SENDER_ID | TEXT | 1 | | 0 |
| 3 | MESSAGE | TEXT | 1 | | 0 |
| 4 | SEEN | INTEGER | 1 | | 0 |

## 3.4 정규화 과정

[제1 정규형] =>

논리적 설계를 통해 산출된 테이블에서 원자 값이 아닌 도메인은 존재하지 않는다 따라서 모든 테이블이 제1 정규형을 만족하고 있다.

[제2 정규형] =>

논리적 설계를 통해 산출된 테이블 중 TOTALLECTURES_DAYS, LECTURES, FRIEND 테이블을 제외한 모든 테이블은 기본키가 한 개의 애트리뷰트로 이루어져 있어 제2정규형을 만족한다. 또한 TOTALLECTURES_DAYS 테이블과 LECTURES 테이블은 애트리뷰트 수 =기본키 수 이므로 제2정규형을 만족하고 FRIEND 테이블은 기본키가 아닌 FRIEND_STATUS가 기본키 (USER1,USER2)에 종속하므로 제2정규형을 만족한다. 따라서 모든 테이블이 제2정규형을 만족한다.

[제3 정규형] =>

MEMBERS 테이블에서 MAJOR_2(학과)를 알면 MAJOR_1(단과대)를 알 수 있는 이행적 함수적 종속성이 존재한다.

따라서 릴레이션 분해를 통해 MEMBERS 테이블을 다음과 같이 변경하였다.

| cid | name | type | notnull | dflt_value | pk |
|---|---|---|---|---|---|
| 0 | ID | TEXT | 0 | | 1 |
| 1 | PW | TEXT | 1 | | 0 |
| 2 | EMAIL | TEXT | 1 | | 0 |
| 3 | REAL_NAME | TEXT | 1 | | 0 |
| 4 | NICK_NAME | TEXT | 1 | | 0 |
| 5 | INTRO | TEXT | 0 | | 0 |
| 6 | YEAR | INTEGER | 0 | | 0 |
| 7 | SEX | TEXT | 0 | | 0 |
| 8 | MAJOR_1 | TEXT | 1 | | 0 |
| 9 | MAJOR_2 | TEXT | 0 | | 0 |

⇩

| cid | name | type | notnull | dflt_value | pk |
|---|---|---|---|---|---|
| 0 | ID | TEXT | 0 | | 1 |
| 1 | PW | TEXT | 1 | | 0 |
| 2 | EMAIL | TEXT | 1 | | 0 |
| 3 | REAL_NAME | TEXT | 1 | | 0 |
| 4 | NICK_NAME | TEXT | 1 | | 0 |
| 5 | INTRO | TEXT | 0 | | 0 |
| 6 | YEAR | INTEGER | 0 | | 0 |
| 7 | SEX | TEXT | 0 | | 0 |
| 8 | MAJOR_2 | TEXT | 1 | | 0 |

| cid | name | type | notnull | dflt_value | pk |
|---|---|---|---|---|---|
| 0 | MAJOR_1 | TEXT | 1 | | 0 |

| | | | | | |
|---|---|---|---|---|---|
| 1 | MAJOR_2 | TEXT | 1 | | 1 |

[BCNF 정규형] =>
논리적 설계를 통해 산출된 테이블에서 모든 결정자가 후보키이다. 따라서 모든 테이블이 BCNF정규형을 만족하고 있다.

## 3.5 물리적 설계(인덱스 등)

아직은 실사용자와 트래픽이 많은 대규모 서비스가 아니므로 일단은 특정 에트리뷰트를 이용해 검색 성능을 높일 수 있는 밀집 인덱스를 이용하기로 하였다. 가장 이용이 많을 것이라고 생각되는 User 테이블과 Lectures 테이블의 인덱스를 다음과 같이 구성하였다. (인덱스 엔트리의 탐색키는 데이터화일의 기본키를 이용하였다.)

MEMBERS table에 대한 인덱스 예시

| | |
|---|---|
| 1 | 레코드 포인터1 |
| 2 | 레코드 포인터2 |
| 3 | 레코드 포인터3 |
| 4 | 레코드 포인터4 |

MEMBERS table 데이터 파일 예시

| | |
|---|---|
| 1 | 데이터1 |
| 2 | 데이터2 |
| 3 | 데이터3 |
| 4 | 데이터4 |

LECTURES table에 대한 인덱스 예시

| | |
|---|---|
| 1 | 레코드 포인터1 |
| 2 | 레코드 포인터2 |
| 3 | 레코드 포인터3 |
| 4 | 레코드 포인터4 |

LECTURES table 데이터 파일 예시

| | |
|---|---|
| 1 | 데이터1 |
| 2 | 데이터2 |

| | |
|---|---|
| 3 | 데이터3 |
| 4 | 데이터4 |

# 4. 설계 결과 및 분석

## 4.1 기능 보완이 필요한 부분 정리 및 개선 방안 제시

해당 어플리케이션은 파이썬 기반의 flask에 sqlite를 이용해 개발하기로 설계하였다. 하지만 sqlite는 데이터베이스 크기가 작고, 현재는 국제캠퍼스 종합강의시간표 데이터만 담고 있어 잘 작동하지만, 서울캠퍼스 종합강의시감표 데이터도 포함한다면 sqlite는 적합하지 않을 것이다. 또한 Web-view 앱으로 기획하였으나 대다수의 호스팅 서비스들이 my sql 등을 지원하였고 sqlite는 지원하는 곳이 거의 없었다. 있다고 하더라도 여러 명의 이용자가 동시 접속 시 속도가 현저히 느려졌다. 따라서 앞으로 서울 캠퍼스까지 서비스를 확대하고, 많은 이용자의 동시 접속에도 무리 없이 서비스를 지원하려면 sqlite가 아닌 다른 DB를 사용해야 할 필요성이 있다.

또한 LECTURES나 MESSAGE 등의 릴레이션은 원래 로그인 시, 회원 id 별로 자료구조를 만들어 개별적으로 데이터를 저장하려 했으나 회원 수 가 많아질수록 릴레이션 수가 급격히 늘어나 용량이 부담이 되어 현재의 방식으로 바꾸었다. 현재는 필요한 값이 있을 때마다 select 문으로 일치하는 id의 튜플을 가져오는 방식을 사용하고 있으나, 회원 수가 늘어난다면 select문을 사용하는 것이 속도 측면에서 불리할 것이다. 따라서 회원 별로 데이터를 모으고 상대적으로 빠르게 해당 데이터를 가져올 수 있는 새로운 방법을 강구해야한다.

## 4.2 설계 목표의 달성 정도를 분석하여 기술

해당 서비스는 이용자가 시간표를 등록, 즉 input이 있지 않으면 동작할 수 없는 서비스이기 때문에, 가장 중요한 시간표 등록, 친구 찾기, 친구 신청 등의 기능을 중점적으로 테스트했다. 또한 DB의 갱신과 수정 이상이 발생하지 않도록 모든 도메인을 원자 값으로 만들고 부분 함수 종속성과 이행적 함수적 종속성 제거하였으며, 결정자가 모두 후보키로 만족시키도록 하여 BCNF 정규형까지 정규화를 완료하였다. 현재까지는 회원 수가 작아 인덱스는 단일단계 인덱스로 구축하였으며 가장 많이 사용되는 애트리뷰트를 인덱스 엔트리의 후보키로 설정하였다. 따라서 논리적 설계, 정규화, 물리적 설계는 설계 목표를 100% 달성했다고 생각한다.

# 5. 결론 및 활용 분야

## 5.1 설계 과정에서의 얻은 여러 교훈들을 나름대로 정리한다.

처음에는 FRIEND릴레이션을 만들 필요성을 느끼지 못하고 MEMBERS릴레이션에 FRIEND_STATUS와 FRIEND_ID 애트리뷰트를 추가하여 '친구'기능을 관리하려 하였다. 하지만 만약 USER1이 USER2, USER3, USER4와 친구 관계를 맺고 있다면, USER1의 튜플이 FRIEND_ID와 FRIEND_STATUS를 포함하여 여러 번 반복되는 것과 같이 중복 데이터가 많이 발생한다. 친구 관계를 맺는 사용자가 많아질수록 중복되는 튜플의 수가 기하급수적으로 증가하게 되어, 데이터베이스의 저장 공간의 비효율성 및 데이터 무결성 조건이 깨지거나 데이터베이스의 성능 저하와 관리의 어려움을 겪을 것이라 예상되었다.

따라서 이를 해결하고자 FRIEND라는 릴레이션을 도입하였다. FRIEND릴레이션을 FRIEND{USER1, USER2, ACTIVE1, ACTIVE2, FRIEND_STATUS}로 설정했었다. ACTIVE1과 ACTIVE2, 그리고 FRIEND_ACTIVE의 기본값을 0으로 초기화하고, 이들의 데이터 타입은 boolean이다.(boolean타입을 적용하지 못할 시 integer타입으로 데이터를 받는다.) FRIEND릴레이션의 USER1가 USER2에게 친구요청을 하면 ACTIVE1이 0->1로 전환되고, USER2가 USER1의 친구요청을 수락시에 ACTIVE2가 0->1로 전환된다. FRIEND_ACTIVE값은 ACTIVE1과 ACTIVE2가 모두 1일 때 0->1로 전환된다. 이 과정은 현재의 과정인 FRIEND_STATUS의 도메인{0, 1, 2} 세 개의 값을 가지는 것과 상반됨을 확인할 수 있다.

최종적으로 변경하기 이전 방식은 세 개의 애트리뷰트를 통해 각 상태 전환을 세분화하여 관리하지만, 더 많은 저장 공간을 요구하게 되어 관리해야 할 로직이 복잡해질 가능성이 있다. 특히 하나의 애트리뷰트(FRIEND_STATUS)를 업데이트하는 것보다 세 개의 애트리뷰트를 각각 관리하고 업데이트하는 것이 성능적으로 비효율적일 수 있다. 따라서 불필요한 애트리뷰트를 줄여나감으로써 FRIEND_STATUS 하나의 애트리뷰트만 사용한다면 '겹강이'의 '친구' 관리 기능을 공간적으로나 성능적으로 더 효율적으로 구현할 것이라 본다. 또한 단일 애트리뷰트 사용으로 쿼리 조건은 더 간단하고 빠르고 최적화에 유리하다고 판단하였다.

또한 '겹강이' 앱을 구현하기 위한 데이터베이스에 정의된 릴레이션 수가 많다보니 릴레이션과 릴레이션 간의 관계를 외래키로 지정하는 것에서 약간의 어려움을 겪었었다. 예를 들어 FRIEND의 USER1과 USER2는 MEMBERS의 ID를 외래키로 가지지만 NOTIFICATION의 SENDER_ID와 USER_ID를 FRIEND의 USER1과 USER2로 각각 맵핑이 되게 외래키로 지정해야 한다. 처음에는 SENDER_ID와 USER_ID를 MEMBERS의 ID로 각각 외래키를 지정하였다. 이처럼 프로젝트 초기 단계에서 요구사항을 명확히 정의하고 분석하는 것이 매우 중요하다는 것을 깨달았다. 초기 설계에서 명확한 요구사항을 설정하지 않으면, 구현 단계에서 예기치 않은 변경 요청이 발생할 수 있으며 이는 프로젝트 일정과 품질에 큰 영향을 미칠 수 있다고 생각했다.

## 5.2 다음 설계 과정에서 고려할 새로운 목표들이 있다면 정리한다.

다수의 사용자가 '겹강이' 앱을 동시에 접속하더라도 빠르고 원활하게 서비스를 이용할 수 있도록 서버 성능을 최적화하려 한다. 또한, 자동 복구 기능과 데이터 백업을 통해 서버의 가용성과 안정성을 높이고자 한다. 그리고 사용자 개인정보를 안전하게 보호할 수 있는 기술을 추가적으로 적용해야 하는 필요성을 느꼈다. 그리고 사용자 수 증가를 대비하여 서버의 확장성을 확보하거나 앞서 4.1에서도 언급했듯이 데이터베이스의 용량과 성능의 유연적 확장에 대한 고려도 불가피하다.

마지막으로 현재 구현된 것보다 데이터베이스 구조를 UI/UX와 더 긴밀하게 연계되도록 설계하여 사용자 편의성을 높이고자 한다. 이를 적용한다면 프론트엔드에서 실시간으로 연동이 더 잘 되어 데이터 로딩 속도를 최적화하고 사용자 대기 시간을 최소화할 수 있다.

## 6. 참고 문헌


[빠르게 활용하는 모바일 데이터베이스 SQLite3]
우상정, 신호철 공저, 위키북스, 2011.05.12

[처음 배우는 플라스크 웹 프로그래밍]
윤정현 저, 한빛미디어, 2021.08.30

[Do it! 점프 투 플라스크]
박응용 저, 이지스퍼블리싱, 2020.11.27

[파이썬(Flask)로 웹사이트 만들기 클론 코딩]
https://www.youtube.com/playlist?list=PLqlc89sXpwUBmr0Z282fm9JurDDYBE55r

[react-native expo webview]
https://docs.expo.dev/versions/latest/sdk/webview/

[react-native expo build]
https://docs.expo.dev/build-reference/apk/